

Projet de Calcul Scientifique

Méthodes d'itération de sous-espaces et applications à la compression
d'images

Sami Chaquir
Aurélien Verbeke
ENSEEIH — 1A SN GH

2024–2025

Table des matières

Introduction	2
1 Étude des méthodes numériques	2
1.1 Limitations de la méthode de la puissance	2
1.2 Méthodes d'itération de sous-espace	3
1.2.1 subspace_iter_v0	3
1.2.2 subspace_iter_v1	4
1.3 Méthodes v2 et v3, plus performantes	4
1.3.1 subspace_iter_v2	4
1.3.2 subspace_iter_v3	5
1.4 Résultats numériques	6
2 Application à la Compression d'Image	8
2.1 Application sur image en couleur	8
Conclusion	10

Introduction

Ce rapport présente les résultats des deux parties du projet de Calcul Scientifique consacré à l'étude des méthodes d'itération de sous-espace, avec une application à la compression d'image. Il s'appuie sur les algorithmes vus en cours (power method, sous-espace, etc.) ainsi que sur leur implémentation en MATLAB.

1 Étude des méthodes numériques

1.1 Limitations de la méthode de la puissance

Question 1 : Comparer le temps d'exécution de `power_v11.m` avec `eig()` sur différents types / tailles de matrices.

Taille matrice	<code>power_v11</code>	<code>eig</code>
100 x 100	0.06	0.01
200 x 200	1.5	0.02
500 x 500	30	0.05

TABLE 1 – Temps d'exécution (s) des méthodes sur différentes tailles de matrices de type `imat = 1`

<code>imat</code>	<code>power_v11</code>	<code>eig</code>
1	0.76	0.02
2	0.01	0.01
3	0.03	0.01
4	0.83	0.02

TABLE 2 – Temps d'exécution des méthodes pour différentes valeurs de `imat` avec matrices de taille 200 x 200

Question 2 : Version optimisée `power_v12.m`

On remplace $y = A \cdot v$; $v = y / \text{norm}(y, 2)$ par $v = z / \text{norm}(z, 2)$

Algorithm 1 Algorithme Power Method Optimisé

```
while norme <  $\epsilon$  and nb_it < maxit do  
     $v \leftarrow z / \|z\|$   
     $z \leftarrow A \cdot v$   
     $\beta \leftarrow v^T z$   
    norme  $\leftarrow \| \beta v - z \| / \|\beta\|$   
    nb_it  $\leftarrow$  nb_it + 1  
end while
```

Comparaison des performances :

Taille matrice	<code>power_v12</code>	<code>eig</code>
100 x 100	0.04	0.01
200 x 200	0.86	0.02
500 x 500	15	0.05

TABLE 3 – Temps d'exécution (s) des méthodes sur différentes tailles de matrices de type `imat = 1` (mêmes matrices qu'à la question 1)

<code>imat</code>	<code>power_v12</code> (s)	<code>eig</code> (s)
1	1.062	1.26×10^{-1}
2	1.56×10^{-2}	1.56×10^{-2}
3	3.125×10^{-2}	4.68×10^{-2}
4	9.22×10^{-1}	1.09×10^{-1}

TABLE 4 – Temps d'exécution des méthodes pour différentes valeurs de "`imat`" avec matrices de taille 200 x 200

Question 3 : Désavantages majeurs de la méthode de la puissance itérée en termes de temps de calcul

Nous observons que le script `power_v12` est environ deux fois plus rapide que `power_v11`, ce qui le rend significativement plus performant et le rapproche de l'efficacité de la fonction MATLAB native `eig`.

Des inconvénients persistent cependant :

- **Convergence lente :** La méthode nécessite un grand nombre d'itérations pour atteindre une précision acceptable, ce qui peut la rendre inefficace pour les grandes matrices.
- **Sensibilité au vecteur initial :** Le choix aléatoire du vecteur initial peut fortement influencer le temps de convergence. Si ce vecteur est mal aligné avec le vecteur propre dominant, le processus peut être considérablement ralenti.
- **Déflation coûteuse :** Lorsqu'on souhaite calculer plusieurs valeurs propres, il est nécessaire d'appliquer une procédure de déflation après chaque calcul. Cette étape ajoute une surcharge de calcul non négligeable.
- **Limitation aux valeurs propres dominantes :** La méthode classique ne permet d'obtenir que la valeur propre de plus grande norme, à moins d'adapter significativement l'algorithme.

1.2 Méthodes d'itération de sous-espace

1.2.1 `subspace_iter_v0`

Question 4 : Vers quelle matrice V converge la méthode de la puissance itérée si on l'applique à un ensemble de m vecteurs ?

Lorsqu'on applique la méthode de la puissance à un ensemble de m vecteurs, le processus converge vers un sous-espace invariant associé aux m plus grandes valeurs propres

(en module) de la matrice A formée des m vecteurs. La matrice V obtenue à la fin de l'algorithme contient donc une base orthonormée de ce sous-espace dominant. Ainsi, V converge vers une matrice dont les colonnes sont les vecteurs propres dominants de A .

Question 5 : Pourquoi calculer la décomposition spectrale complète de H n'est pas un problème ?

Les vecteurs propres de A associés aux valeurs propres dominantes s'obtiennent alors par $V_{\text{out}} = VX$, avec X la décomposition spectrale de H .

Or $H \in \mathbb{R}^{m \times m}$ est une petite matrice puisque généralement $m \ll n$. Ainsi, le coût de calcul de la décomposition spectrale de H reste négligeable comparé au coût global de l'algorithme, et de ce qu'il aurait pu être en calculant la décomposition spectrale de A .

Cela permet donc d'extraire assez efficacement les m valeurs propres dominantes de A .

1.2.2 subspace_iter_v1

Question 7 : Identifier toutes les étapes de l'algorithme dans le fichier `subspace_iter_v1.m`.

Algorithm 2 Subspace iteration method v1 with Raleigh-Ritz projection

Require: Symmetric matrix $A \in \mathbb{R}^{n \times n}$, tolerance ε , *MaxIter* (max nb of iterations) and *PercentTrace* the target percentage of the trace of A

Ensure: n_{ev} dominant eigenvectors V_{out} and the corresponding eigenvalues Λ_{out} .

```

Generate an initial set of  $m$  orthonormal vectors  $V \in \mathbb{R}^{n \times m}$ ;  $k = 0$ ; PercentReached =
0                                     ▷ Lignes 38, 40, 42, 48, 49
repeat                               ▷ Ligne 52
     $k = k + 1$                            ▷ Ligne 54
    Compute  $Y$  such that  $Y = A \cdot V$     ▷ Ligne 56
     $V \leftarrow$  orthonormalisation of the columns of  $Y$     ▷ Ligne 58
    Rayleigh-Ritz projection applied on matrix  $A$  and orthonormal vectors  $V$     ▷ Ligne
61
    Convergence analysis step : save eigenpairs that have converged and update
PercentReached                               ▷ Lignes 70 à 109
until ( PercentReached > PercentTrace or  $n_{ev} = m$  or  $k > \text{MaxIter}$  )    ▷ Ligne 117

```

1.3 Méthodes v2 et v3, plus performantes

1.3.1 subspace_iter_v2

Question 8 : Quel est le coût en terme de nombre de calculs du calcul de A^p et $A^p V$. Comment modifier le calcul pour l'optimiser ?

Rappel : L'algorithme naïf de produit matriciel sur une matrice de taille $n \times n$ est de complexité $\mathcal{O}(n^3)$.

A^p

Si A est une matrice symétrique de taille $n \times n$ et p un entier strictement positif, alors le calcul de A^p nécessite $p - 1$ produits matriciels successifs, ce qui implique une complexité en $\mathcal{O}((p - 1)n^3)$.

$A^p \cdot V$

On ajoute une multiplication matricielle.

La complexité globale de cette étape est donc en $\mathcal{O}(pn^3)$.

Pour optimiser le calcul, on ne calcule qu'une seule fois A^p avant les itérations.

Question 10 : Observer le comportement de cette méthode à l'augmentation de p .

Méthode	p	Nombre d'itérations
v0		2090
v1		263
v2	2	132
v2	5	53
v2	10	27
v2	20	14
v2	30	9
v2	50	6
v2	100	3
v2	110	3
v2	120	3
v2	130	3
v2	140	Erreur : pourcentage non atteint

TABLE 5 – Nombre d'itérations des méthodes sur une matrice de taille 200×200 et de type `imat = 1` (même matrice que précédemment)

Lorsque p augmente, le temps de calcul diminue simplement. Au bout d'un certain moment, on plafonne puis le calcul n'aboutit plus.

1.3.2 subspace_iter_v3

Question 11 : Expliquer pourquoi la qualité des couples propres diffère pour quelques vecteurs (pour `subspace_iter_v1` et `subspace_iter_v2`).

Dans les algorithmes `v1` et `v2`, on fait converger l'ensemble des erreurs à chaque itération. Ainsi, On fait converger des vecteurs qui ont déjà convergé. Donc les premiers couples propres seront nécessairement plus proches du résultat.

En effet, on observe aisément un rapport de qualité de 10^5 entre les premiers couples propres et les derniers.

Question 12 : Prédire le comportement pour `subspace_iter_v3`.

Comme on ne fait plus converger les vecteurs qui ont déjà convergé, on devrait atteindre une précision des couples propres équivalente pour chacun.

C'est en effet ce qui est obtenu au lancement du programme.

1.4 Résultats numériques

Question 14 : Quelles sont les différences des 4 types de matrices ?

La différence des 4 types de matrices est la distribution de leurs valeurs propres :

- **Type 1 :** Valeurs propres réparties linéairement de n à 1.
- **Type 2 :** Valeurs propres aléatoires entre $\frac{1}{\text{cond}}$ et 1 dont les logarithmes sont uniformément répartis. Avec $\text{cond} = 10^{10}$.
- **Type 3 :** Valeurs propres égales à $\text{cond}^{\frac{i}{n-1}}$. Avec $i \in [0; n-1]$ et $\text{cond} = 10^5$.
- **Type 4 :** Valeurs propres égales à $1 - \frac{i}{n-1} \times (1 - \frac{1}{\text{cond}})$. Avec $i \in [0; n-1]$ et $\text{cond} = 10^2$. Donc réparties linéairement entre 1 et $\frac{1}{\text{cond}}$.

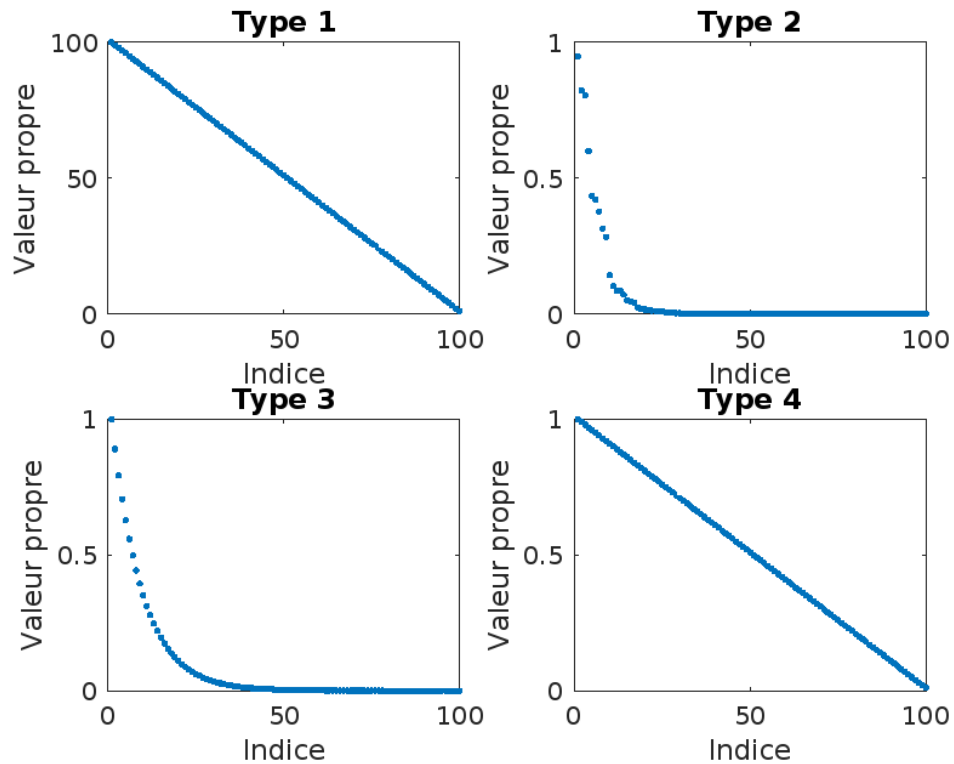


FIGURE 1 – Distributions spectrales des différents types de matrice

Question 15 : Comparer les performances des algorithmes codés et fournis (dont `eig`) pour différents types et tailles de matrices.

Taille matrice	Type matrice (imat)	<code>eig</code>	<code>v11</code>	<code>v12</code>	<code>v0</code>	<code>v1</code>	<code>v2</code>	<code>v3</code>
100x100	1	0.02	0.07	0.09	2.4	0.38	0.06	0.07
	2	< 0.01	< 0.01	< 0.01	0.07	0.06	N/A	N/A
	3	0.01	< 0.01	0.01	0.36	0.09	0.79	1.10
	4	0.01	0.08	0.06	2.00	0.37	0.09	0.08
200x200	1	0.02	2.30	1.40	4.70	8.20	0.08	0.09
	2	0.01	0.03	0.02	0.27	0.09	0.15	0.15
	3	0.01	0.09	0.06	0.85	0.12	1.50	1.40
	4	0.02	2.49	1.38	5.52	0.97	0.09	0.11
300x300	1	0.03	6.76	4.60	8.33	2.78	0.12	0.14
	2	0.03	0.08	0.06	2.00	0.10	0.11	0.14
	3	0.03	0.25	0.18	1.61	0.14	0.13	0.14
	4	0.04	9.17	2.94	6.83	3.05	0.15	0.16
500x500	1	0.46	N/A	N/A	27.8	N/A	N/A	N/A
	2	0.44	0.57	0.39	1.39	0.20	0.98	1.23
	3	0.12	1.43	1.16	3.26	0.40	0.14	0.13
	4	0.11	N/A	N/A	31.1	N/A	N/A	N/A

TABLE 6 – Vitesses d’exécutions (s) pour le calcul de 20 couples propres ou un pourcentage d’au moins 10% du nombre de couples propres, facteur d’accélération $p = 50$

N/A : Convergence non atteinte

2 Application à la Compression d'Image

Question 1 : Tailles des matrices du triplet (Σ_k, U_k, V_k)

$\Sigma_k \in \mathbb{R}^{k \times k}$: matrice diagonale contenant les k plus grandes valeurs singulières σ_i

$U_k \in \mathbb{R}^{q \times k}$: k premiers vecteurs singuliers à gauche

$V_k \in \mathbb{R}^{p \times k}$: k premiers vecteurs singuliers à droite

Si $q < p$, on intervertira les rôles de U et V , et on construira $M = II^T$ plutôt que $I^T I$.

Question 2 : Comparer les différentes fonctions sur leur application à la compression d'images.

Erreur RMSE (Root Mean Square Error) :

Pour une matrice (image) I et son approximation I_k de rang k

$$\text{RMSE} = \sqrt{\sum_{i,j} (I_{ij} - I_{kij})^2}$$

Méthode	Temps (s)	RMSE	k utilisé
eig	0.23	5.71	50
power method	1.78	5.85	50
subspace iter (v1)	0.54	5.73	50
subspace iter (v2)	0.48	5.71	50
subspace iter (v3)	0.41	5.70	50
subspace iter (v4)	0.38	5.70	50

TABLE 7 – Comparaison des méthodes de calcul des valeurs singulières

Commentaires : les variantes avancées de l'itération de sous-espace montrent des performances supérieures, tant en rapidité qu'en précision. La méthode classique `eig` reste toutefois une référence pour la comparaison.

2.1 Application sur image en couleur

L'image en couleur est représentée par trois canaux (R, G, B). Chaque canal est compressé indépendamment par la méthode de SVD. On reconstruit ensuite l'image couleur en combinant les trois matrices reconstruites.



FIGURE 2 – Résultat de la compression sur une image en couleur

Conclusion

Ce projet nous a permis d'explorer plusieurs méthodes de calcul d'approximation de rang faible via la SVD. L'approche par itération de sous-espace s'est montrée très efficace, en particulier les versions avec orthonormalisation. La compression d'images est bien illustrée par la conservation de la structure globale de l'image même après forte réduction de rang. Enfin, l'extension à la couleur démontre la robustesse de l'approche matricielle en traitement d'image.